# ENCLAVE — Product Requirements Document

> **For Claude Code** · Phase-by-phase implementation guide · March 2026
>
> Stack: Next.js 15 · PostgreSQL + Prisma · NextAuth v5 · Stripe · Dokploy · OpenClaw · lossless-claw · Qdrant · Mem0
>
> Repo: `github.com/mtldev-dotcom/clawhost` · Production: `nestai.nickybruno.com` · Dokploy: `35.202.32.236:3000`

---

## How to Read This Document

Each phase is **self-contained**. Within each phase, steps must be completed in order — later steps depend on earlier ones. Each step has:

- **Goal** — what to build
- **Files to create or modify**
- **Acceptance criteria** — how to verify it works
- **Never do** — common mistakes to avoid

Execute autonomously. Only pause if a secret/credential is missing from `.env.local`.

---

## Current State (Before Phase 1)

What already exists and works:

- Next.js 15 app deployed at `nestai.nickybruno.com` via Dokploy
- PostgreSQL at `35.202.32.236:5432/nestai`
- NextAuth v5 with register/login pages
- Stripe checkout + webhook handler (route exists, not yet fully tested)
- Dokploy provisioning: `POST /api/provision` creates an OpenClaw container per user
- OpenClaw provisions with `TELEGRAM_BOT_TOKEN` + `OPENAI_API_KEY`

- 5-step onboarding wizard (API key validation + Telegram pairing)
- Bilingual UI (EN/FR via next-intl)
- Vitest unit tests (34 tests) + Playwright E2E

What does NOT exist yet:

- lossless-claw on provisioned instances
- Persistent volumes in Dokploy
- Workspace / file storage
- Any third-party integrations
- Skills marketplace
- Memory dashboard

---

## Environment Variables Reference

```
# Platform
DATABASE_URL=postgresql://...@35.202.32.236:5432/nestai
NEXTAUTH_SECRET=...
NEXTAUTH_URL=https://nestai.nickybruno.com
AUTH_TRUST_HOST=true

# Stripe
STRIPE_SECRET_KEY=sk_...
STRIPE_WEBHOOK_SECRET=whsec_...
STRIPE_PRICE_ID_STARTER=price_...
STRIPE_PRICE_ID_PRO=price_...
STRIPE_PRICE_ID_BUILDER=price_...

# Dokploy
DOKPLOY_URL=http://35.202.32.236:3000
DOKPLOY_API_KEY=...

# Provisioning defaults
OPENCLAW_IMAGE=ghcr.io/openclaw/openclaw:latest
OPENCLAW_BASE_DOMAIN=nickybruno.com
```

---

---

# PHASE 1 — Memory Foundation + Stable Core

**Goal:** Every provisioned OpenClaw instance gets lossless-claw, persistent storage, and encrypted-at-rest memory. The never-forget promise becomes technically real.

**Timeline:** Now → Month 1

---

## Step 1.1 — Persistent Volume Support in Dokploy Provisioning

### Goal

When a new user is provisioned, their OpenClaw container gets a named Docker volume that survives container restarts. This is the physical foundation for all memory storage.

### Files to modify

`src/lib/dokploy.ts`

### What to change

In the `provisionInstance` function (or equivalent), update the Docker Compose template passed to Dokploy as `sourceType: "raw"` to include a named volume:

```
# Add to the compose template string
services:
  openclaw:
    image: ${OPENCLAW_IMAGE}
    restart: unless-stopped
    environment:
      TELEGRAM_BOT_TOKEN: "${telegramBotToken}"
      OPENAI_API_KEY: "${openaiApiKey}"
      ANTHROPIC_API_KEY: "${anthropicApiKey}"
      # Memory layer config
      LCM_ENABLED: "true"
      LCM_DATABASE_PATH: "/data/lcm.db"
      LCM_FRESH_TAIL_COUNT: "32"
      LCM_INCREMENTAL_MAX_DEPTH: "1"
      LCM_CONTEXT_THRESHOLD: "0.75"
      # Encryption key (unique per user, generated at provision time)
      ENCLAVE_ENCRYPTION_KEY: "${encryptionKey}"
    volumes:
      - openclaw-data:/data
    ports:
```

```
      - "18789:18789"

volumes:
  openclaw-data:
    name: "enclave-${userId}"
    driver: local
```

## What to add to provisioning logic

In `src/lib/dokploy.ts`, generate a unique encryption key at provision time:

```
import { randomBytes } from 'crypto'

// Inside provisionInstance():
const encryptionKey = randomBytes(32).toString('hex')

// Store ONLY the key reference (not the key itself) in our DB
// The key lives only in the container env var
await prisma.instance.update({
  where: { userId },
  data: {
    volumeName: `enclave-${userId}`,
    provisionedAt: new Date(),
    // DO NOT store encryptionKey here
  }
})
```

## Database migration needed

```
// prisma/schema.prisma — add to Instance model:
volumeName    String?
lcmEnabled    Boolean @default(false)
```

Run: `npx prisma migrate dev --name add_volume_fields`

## Acceptance criteria

- New provision creates a `docker volume ls` entry named `enclave-{userId}`
- Container restart does not lose `/data` contents
- `encryptionKey` is NEVER written to the `instances` table in PostgreSQL
- `ENCLAVE_ENCRYPTION_KEY` env var is present in provisioned container

## Step 1.2 — Install lossless-claw on All Instances

### Goal

Every provisioned OpenClaw instance runs lossless-claw as its context engine. The agent never truncates old messages — everything goes into the SQLite DAG on the persistent volume.

### Files to modify

`src/lib/dokploy.ts` — update the OpenClaw config injected at provision time

### What to change

The OpenClaw config file ( `openclaw.json` ) must be injected into the container at startup. Add it to the compose template as a mounted config or via environment variable:

```
# In compose template, add environment var for OpenClaw config:
environment:
  # ... existing vars ...
  OPENCLAW_CONFIG: |
    {
      "agents": {
        "defaults": {
          "model": "${model}",
          "workspace": "/data/workspace"
        }
      },
      "plugins": {
        "slots": {
          "contextEngine": "lossless-claw"
        },
        "config": {
          "lossless-claw": {
            "enabled": true,
            "freshTailCount": 32,
            "contextThreshold": 0.75,
            "incrementalMaxDepth": 1
          }
        }
      }
    }
```

> **Note:** OpenClaw reads `OPENCLAW_CONFIG` as a JSON string if the config file path is not

> found. Verify this against current OpenClaw docs — if not supported, use a startup script instead (see fallback below).

**Fallback approach** — if OPENCLAW_CONFIG env var is not supported, inject a startup script:

```
# In compose template:
entrypoint: ["/bin/sh", "-c"]
command:
  - |
    mkdir -p /root/.openclaw
    cat > /root/.openclaw/openclaw.json << 'EOF'
    {
      "agents": { "defaults": { "model": "${model}", "workspace": "/data/workspac
      "plugins": {
        "slots": { "contextEngine": "lossless-claw" },
        "config": { "lossless-claw": { "enabled": true, "freshTailCount": 32, "co
      }
    }
    EOF
    openclaw plugins install @martian-engineering/lossless-claw --yes
    openclaw gateway --port 18789
```

## Post-provision verification endpoint

Create `src/app/api/admin/verify-instance/route.ts`:

```
// GET /api/admin/verify-instance?userId=xxx
// Checks that lossless-claw is running on the instance
// Only accessible by admins (check session role)
export async function GET(req: Request) {
  // 1. Get instance URL from DB
  // 2. Hit the OpenClaw health endpoint
  // 3. Return lcm status
}
```

## Database migration

```
// Add to Instance model:
lcmEnabled    Boolean @default(false)
lcmInstalledAt DateTime?
```

Run: `npx prisma migrate dev --name add_lcm_fields`

After successful install, set `lcmEnabled: true` on the instance record.

### Acceptance criteria

- New instance: `openclaw plugins list` shows `lossless-claw` as enabled
- After 3+ messages, `/data/lcm.db` exists and has non-zero size
- `/compact` command in Telegram routes through lossless-claw (check logs)
- Container restart: `lcm.db` survives (volume persists)
- `lcmEnabled: true` written to DB after successful provision

---

## Step 1.3 — Stripe Webhook End-to-End Test + Hardening

### Goal

The full flow — user registers → pays → auto-provisions with persistent volume + lossless-claw → gets Telegram bot — must work reliably in production.

### Files to audit/fix

- `src/app/api/stripe/webhook/route.ts`
- `src/app/api/provision/route.ts`
- `src/lib/stripe.ts`

### Checklist

**Webhook handler must:**

1. Verify `stripe-signature` header using `STRIPE_WEBHOOK_SECRET`
2. Handle `checkout.session.completed` event
3. Extract `userId` from session metadata
4. Call `provisionInstance(userId)` — which now includes volume + lossless-claw
5. Update `instance.status = 'provisioning'` immediately
6. Handle idempotency — if provision already exists for userId, skip
7. Return 200 immediately (Stripe retries on non-200)

**Error handling:**

- If provision fails, set `instance.status = 'failed'` and alert (log + optionally email)
- Stripe will retry webhook up to 3 days — handle duplicate events gracefully

**Test in production:**

```
# Install Stripe CLI if not present
stripe listen --forward-to https://nestai.nickybruno.com/api/stripe/webhook
stripe trigger checkout.session.completed
```

### Acceptance criteria

- Stripe CLI test triggers full provision flow
- User in DB gets `instance.status = 'active'` within 60 seconds
- Duplicate webhook event does not create a second container
- Failed provision sets `status = 'failed'` (not left as `'provisioning'` forever)

---

## Step 1.4 — Dashboard: Instance Status Panel

### Goal

Users can see their agent's current status from the dashboard. Replaces any placeholder UI.

### Files to create/modify

- `src/app/dashboard/page.tsx` — main dashboard view
- `src/app/api/instance/status/route.ts` — polling endpoint

### Status endpoint

```
// GET /api/instance/status
// Returns current instance state for the logged-in user
export async function GET() {
  const session = await auth()
  const instance = await prisma.instance.findUnique({
    where: { userId: session.user.id },
    select: {
```

```
      status: true,
      subdomain: true,
      telegramBotUsername: true,
      lcmEnabled: true,
      provisionedAt: true,
    }
  })
  return Response.json(instance)
}
```

### Dashboard UI states to handle

1. **No subscription** → show pricing / upgrade CTA
2. **Provisioning** → spinner + "Your agent is being set up..."
3. **Failed** → error message + support contact
4. **Active** → show agent URL, Telegram link, lossless-claw status badge

### Acceptance criteria

- Dashboard shows correct status for each state
- `lcmEnabled: true` shows a "Memory: Active" badge
- Auto-refreshes every 10s when status is `'provisioning'`

---

## Step 1.5 — DNS + Production Verification

### Goal

Confirm all DNS records point to the new IP and all user subdomains resolve correctly.

### DNS records required

```
*.nickybruno.com          A    35.202.32.236
dok.nestai.nickybruno.com A    35.202.32.236
nestai.nickybruno.com     A    35.202.32.236
```

### Verification script

Create `scripts/verify-dns.sh`:

```
#!/bin/bash
echo "=== DNS Verification ==="
dig +short nestai.nickybruno.com
dig +short dok.nestai.nickybruno.com
dig +short test.nickybruno.com
echo "Expected: 35.202.32.236"
```

### Acceptance criteria

- All three DNS records resolve to `35.202.32.236`
- `https://nestai.nickybruno.com` loads without SSL errors
- `http://35.202.32.236:3000` (Dokploy panel) is accessible

---

## Phase 1 Definition of Done

- New user signup → Stripe payment → Telegram bot working in < 2 minutes
- lossless-claw running on all new instances
- Persistent volumes prevent data loss on container restart
- Encryption key generated at provision, never stored in DB
- Dashboard shows live instance status
- All existing Vitest tests still pass ( `npm test` )
- No regressions on auth flow, onboarding wizard

---

---

# PHASE 2 — Workspace + Web Chat + Google Integrations

**Goal:** Users get a private file workspace, an in-browser chat, and their first real integrations (Google Calendar + Gmail).

**Timeline:** Month 2

**Prerequisite:** Phase 1 complete.

## Step 2.1 — Enclave Drive (Workspace File System)

### Goal

Every user gets a private file storage area in the dashboard. Both the user (via UI) and the agent (via MCP tool) can read and write files here. Files are stored on the user's persistent volume.

### Database schema

```
model WorkspaceFile {
  id          String   @id @default(cuid())
  userId      String
  user        User     @relation(fields: [userId], references: [id], onDelete: Ca
  name        String
  path        String   // relative path within workspace, e.g. "projects/atlas/br
  mimeType    String
  sizeBytes   Int
  content     String?  // for text files < 100kb, store inline
  storageKey  String   // path on persistent volume: /data/workspace/{userId}/{st
  folderId    String?
  folder      WorkspaceFolder? @relation(fields: [folderId], references: [id])
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  createdBy   String   // 'user' | 'agent'
  description String?  // agent-generated summary of file contents
  indexed     Boolean  @default(false) // has been vectorized for search

  @@index([userId])
}

model WorkspaceFolder {
  id        String            @id @default(cuid())
  userId    String
  name      String
  parentId  String?
  parent    WorkspaceFolder?  @relation("FolderTree", fields: [parentId], referen
  children  WorkspaceFolder[] @relation("FolderTree")
  files     WorkspaceFile[]
  createdAt DateTime          @default(now())

  @@index([userId])
}
```

Run: `npx prisma migrate dev --name add_workspace`

## API routes

### File upload:

```
// POST /api/workspace/upload
// multipart/form-data: file, folderId?
// 1. Validate file (type, size < 50MB)
// 2. Generate storageKey = crypto.randomUUID() + extension
// 3. Write to /data/workspace/{userId}/{storageKey} on volume via SSH or direct
// 4. Create WorkspaceFile record
// 5. Trigger background indexing job (async)
```

### File list:

```
// GET /api/workspace/files?folderId=&search=
// Returns files for the logged-in user
// Full-text search across name + description if search param present
```

### File download:

```
// GET /api/workspace/files/[id]/download
// Stream file content from persistent volume
```

### File delete:

```
// DELETE /api/workspace/files/[id]
// Soft-delete: set deletedAt, don't remove from volume immediately
```

## Dashboard UI

Create `src/app/dashboard/workspace/page.tsx`:

Layout:

```
┌──────────────────────────────────────────┐
│  📁 Workspace                   [+ Upload] │
├──────────┬──────────────────────────────┤
│ Folders  │  Files list (name, size, date,  │
│ tree     │  created by user/agent)          │
│          │                                  │
│ 📁 Inbox │  [file rows with preview icons]  │
│ 📁 Proj. │                                  │
```

```
│ 📁 Notes │                                  │
└──────────┴──────────────────────────────────┘
```

Components to build:

- `src/components/workspace/FileUpload.tsx` — drag-and-drop + click
- `src/components/workspace/FileList.tsx` — sortable list with type icons
- `src/components/workspace/FolderTree.tsx` — sidebar nav
- `src/components/workspace/FilePreview.tsx` — modal for PDF/image preview

### MCP tool for agent access

Create `src/lib/mcp/workspace-tool.ts`:

This is an MCP server the agent calls to access workspace files. It exposes:

- `workspace_list(folder?)` — list files
- `workspace_read(fileId)` — get file content
- `workspace_write(name, content, folder?)` — create/update a file
- `workspace_search(query)` — full-text search

The MCP server runs as a sidecar in the user's OpenClaw container. Add it to the compose template in `dokploy.ts`.

### Acceptance criteria

- User can upload PDF, DOCX, images, CSV, TXT from dashboard
- Files persist across browser sessions
- Agent can call `workspace_read` and `workspace_write` via MCP
- Files created by agent show "Created by: Agent" label in UI
- Search returns results across file names and descriptions
- 50MB file size limit enforced with clear error message

---

## Step 2.2 — Web Chat in Dashboard

## Goal

Users can chat with their agent directly from the browser, not just via Telegram.

## How it works

OpenClaw has a WebChat surface that runs on port 18789 (the Gateway WebSocket). The dashboard embeds this via an iframe or proxied WebSocket connection.

## Files to create

- `src/app/dashboard/chat/page.tsx`
- `src/components/dashboard/ChatEmbed.tsx`

## Implementation

Option A — Direct iframe (simplest):

```
// ChatEmbed.tsx
export function ChatEmbed({ instanceUrl }: { instanceUrl: string }) {
  return (
    <iframe
      src={`${instanceUrl}/chat`}
      className="w-full h-full border-0 rounded-lg"
      allow="microphone"
    />
  )
}
```

Option B — Proxied WebSocket (more secure, hides direct instance URL):

```
// src/app/api/ws/chat/route.ts
// WebSocket proxy: client ↔ Enclave gateway ↔ OpenClaw gateway
// Validates session before proxying
// Hides the raw instance URL from the browser
```

Prefer Option B for production. Implement Option A first, then upgrade.

## Dashboard navigation

Add “Chat” as the primary dashboard route ( `/dashboard` → redirect to `/dashboard/chat` ).

## Acceptance criteria

- User can send a message from the browser and get a response

- Chat history loads on page refresh (from lossless-claw via OpenClaw)
- Works on mobile (responsive layout)
- Chat and Telegram are the same agent (same memory, same session or linked sessions)

---

## Step 2.3 — Google Calendar Integration

### Goal

The agent can read and write to the user's Google Calendar. Users connect their Google account once from the settings page; the agent handles the rest.

### OAuth flow

1. User clicks "Connect Google Calendar" in dashboard settings
2. Redirect to Google OAuth with scopes: `calendar.readonly`, `calendar.events`
3. Google redirects back to `/api/integrations/google/callback`
4. Store refresh token (encrypted) in DB
5. Agent can now call Calendar API on user's behalf

### Database schema

```
model Integration {
  id           String   @id @default(cuid())
  userId       String
  user         User     @relation(fields: [userId], references: [id], onDelete: C
  provider     String   // 'google', 'notion', 'github', etc.
  accessToken  String   // encrypted
  refreshToken String   // encrypted
  tokenExpiry  DateTime?
  scopes       String[] // granted scopes
  metadata     Json?    // provider-specific config
  connected    Boolean  @default(true)
  connectedAt  DateTime @default(now())
  lastUsedAt   DateTime?

  @@unique([userId, provider])
  @@index([userId])
}
```

Run: `npx prisma migrate dev --name add_integrations`

## Token encryption

```
// src/lib/crypto.ts
import { createCipheriv, createDecipheriv, randomBytes } from 'crypto'

const ALGORITHM = 'aes-256-gcm'
const KEY = Buffer.from(process.env.TOKEN_ENCRYPTION_KEY!, 'hex') // 32 bytes

export function encryptToken(plaintext: string): string {
  const iv = randomBytes(16)
  const cipher = createCipheriv(ALGORITHM, KEY, iv)
  const encrypted = Buffer.concat([cipher.update(plaintext, 'utf8'), cipher.final
  const authTag = cipher.getAuthTag()
  return [iv, authTag, encrypted].map(b => b.toString('hex')).join(':')
}

export function decryptToken(ciphertext: string): string {
  const [ivHex, authTagHex, encryptedHex] = ciphertext.split(':')
  const iv = Buffer.from(ivHex, 'hex')
  const authTag = Buffer.from(authTagHex, 'hex')
  const encrypted = Buffer.from(encryptedHex, 'hex')
  const decipher = createDecipheriv(ALGORITHM, KEY, iv)
  decipher.setAuthTag(authTag)
  return Buffer.concat([decipher.update(encrypted), decipher.final()]).toString('
}
```

Add `TOKEN_ENCRYPTION_KEY` to `.env.example` (generate with `openssl rand -hex 32`).

## API routes

```
// GET  /api/integrations/google/connect     — initiate OAuth
// GET  /api/integrations/google/callback    — handle OAuth callback
// DELETE /api/integrations/[provider]       — disconnect integration
// GET  /api/integrations                    — list connected integrations
```

## MCP tool for agent

Create `src/lib/mcp/google-calendar-tool.ts` — exposes:

- `calendar_list_events(start, end, calendarId?)` — list events in range
- `calendar_create_event(title, start, end, description?, attendees?)` — create

event

- `calendar_update_event(eventId, changes)` — update event
- `calendar_delete_event(eventId)` — delete event
- `calendar_find_free_slots(duration, within)` — find availability

The MCP server is injected into the user's OpenClaw container alongside their credentials (via env var, not file).

### Settings UI

Create `src/app/dashboard/settings/integrations/page.tsx`:

```
┌──────────────────────────────────────────┐
│  Integrations                            │
├──────────────────────────────────────────┤
│  [Google] Google Calendar    [✓ Connected]  │
│  [Google] Gmail              [Connect]      │
│  [Notion] Notion             [Connect]      │
│  [GitHub] GitHub             [Connect]      │
└──────────────────────────────────────────┘
```

### Acceptance criteria

- OAuth flow completes and stores encrypted refresh token
- Agent can list today's events via `calendar_list_events`
- Agent can create an event when asked "schedule a meeting tomorrow at 2pm"
- Token refresh works silently when access token expires
- Disconnecting removes the token and disables the MCP tool
- No Google credentials ever logged or exposed in API responses

---

## Step 2.4 — Gmail Integration

### Goal

The agent can read, draft, and send email on the user's behalf. Uses Gmail Pub/Sub for real-time inbox monitoring.

### OAuth scopes needed

`gmail.readonly`, `gmail.send`, `gmail.modify`, `gmail.labels`

### Implementation approach

Gmail integration shares the Google OAuth flow from Step 2.3. If the user already connected Google Calendar, prompt them to re-authorize with the additional Gmail scopes.

### MCP tools

- `gmail_list_messages(query?, maxResults?)` — list/search messages
- `gmail_get_message(messageId)` — get full message content
- `gmail_send(to, subject, body, cc?, bcc?)` — send email
- `gmail_draft(to, subject, body)` — create draft (no send)
- `gmail_reply(messageId, body)` — reply to a thread
- `gmail_archive(messageId)` — archive
- `gmail_label(messageId, label)` — apply label

### Gmail Pub/Sub (real-time inbox monitoring)

OpenClaw has native Gmail Pub/Sub support. Configure it per-instance:

```
// In dokploy.ts, add to openclaw.json config template:
"automation": {
  "gmail": {
    "enabled": true,
    "pubsubTopic": "projects/enclave-prod/topics/gmail-{userId}",
    "watchLabels": ["INBOX", "UNREAD"]
  }
}
```

This lets the agent react to new emails in real time without polling.

### Acceptance criteria

- Agent can summarize the last 10 unread emails when asked
- Agent can draft a reply and show it for approval before sending
- Agent can send email when explicitly approved by user

- Gmail Pub/Sub fires agent on new inbox messages (test with a real email)

---

## Step 2.5 — Morning Briefing Skill (First Cron Skill)

### Goal

The first automated skill. Every morning at a user-configured time, the agent sends a briefing to Telegram: today's calendar events, unread email count and priority items, open tasks (from workspace notes), and a daily focus suggestion.

### Database schema

```
model UserSkill {
  id        String   @id @default(cuid())
  userId    String
  user      User     @relation(fields: [userId], references: [id], onDelete: Casc
  skillId   String   // 'morning-briefing', 'inbox-zero', etc.
  enabled   Boolean  @default(true)
  config    Json     // skill-specific config (e.g., { "time": "07:30", "timezone
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  @@unique([userId, skillId])
}
```

Run: `npx prisma migrate dev --name add_skills`

### Skill implementation

OpenClaw has native cron support. The Morning Briefing skill is a cron job + prompt file injected into the user's workspace.

Create `src/lib/skills/morning-briefing.ts`:

```
export function getMorningBriefingSkillConfig(config: {
  time: string      // "07:30"
  timezone: string  // "America/Montreal"
}) {
  return {
    cron: {
      schedule: `${config.time.split(':')[1]} ${config.time.split(':')[0]} * * *`
      timezone: config.timezone,
      task: `
```

```
        Prepare and send a morning briefing to the user via Telegram.

        Include:
        1. Today's calendar events (use calendar_list_events for today)
        2. Unread email count and top 3 priority items (use gmail_list_messages)
        3. Any open tasks from workspace notes
        4. One focus recommendation based on the calendar and tasks

        Keep it concise. Use emoji for scannability.
        If it's a light day (< 2 meetings), keep it short.
        If it's a packed day (> 4 meetings), add prep notes for key meetings.
      `
    }
  }
}
```

### Settings UI for Morning Briefing

Add to `src/app/dashboard/settings/skills/page.tsx`:

```
Morning Briefing
[Toggle: ON]
Time: [07:30]  Timezone: [America/Montreal ▼]
[Save]
```

### Acceptance criteria

- User can enable Morning Briefing with configurable time
- Cron fires at the correct local time (test with a 1-minute future time)
- Briefing includes calendar events pulled from Google Calendar
- Briefing is sent to the user's Telegram

---

## Phase 2 Definition of Done

- Workspace file upload, list, download working
- Agent can write files to workspace via MCP tool
- Web Chat loads in dashboard and connects to the same agent as Telegram
- Google Calendar: OAuth connected, agent can create/read events

- Gmail: OAuth connected, agent can read and draft emails
- Morning Briefing skill fires on schedule
- Integration settings page shows connected/disconnected status
- All Phase 1 tests still pass

---

---

# PHASE 3 — Semantic Memory + Full Knowledge Layer

**Goal:** Add Qdrant vector search (Layer 2 memory), Mem0 entity extraction (Layer 3 memory), and the Memory Dashboard UI. Also add Notion + Google Sheets + TickTick integrations.

**Timeline:** Month 3

**Prerequisite:** Phase 2 complete.

---

## Step 3.1 — Qdrant Sidecar per User Instance

### Goal

Each user's OpenClaw container gets a Qdrant vector database sidecar. All messages are vectorized and stored for semantic (meaning-based) search.

### Architecture

```
User's Docker stack (per user):
┌──────────────────────────────────────┐
│  openclaw container  (port 18789)    │
│  qdrant container    (port 6333)     │
│  shared volume: enclave-{userId}     │
└──────────────────────────────────────┘
```

### Files to modify

`src/lib/dokploy.ts` — update compose template

```
services:
  openclaw:
    # ... existing config ...
    environment:
      # ... existing vars ...
      QDRANT_URL: "http://qdrant:6333"
    depends_on:
      - qdrant

  qdrant:
    image: qdrant/qdrant:latest
    restart: unless-stopped
    volumes:
      - openclaw-data:/qdrant/storage
    # No external port exposure — only accessible within Docker network
```

> Qdrant stores its data at `/qdrant/storage` — this shares the same named volume so it persists alongside lossless-claw's SQLite database.

### MCP tool for semantic search

Create `src/lib/mcp/qdrant-tool.ts`:

- `memory_search(query, limit?)` — semantic search across all vectorized content
- `memory_store(content, metadata)` — manually store a piece of content

### Vectorization pipeline

After each conversation turn, lossless-claw ingests the message. Add a post-ingest hook that:

1. Takes the message content
2. Generates an embedding via the user's configured LLM provider
3. Upserts the embedding into Qdrant with metadata (timestamp, session, message_id)

This hook can be implemented as a lossless-claw config option or as a separate OpenClaw skill that watches for new messages.

### Acceptance criteria

- `docker ps` on the host shows a `qdrant` container per user
- After 10+ messages, Qdrant collection has 10+ vectors

- Agent can find a message by meaning using `memory_search`
- "Find that conversation about the Montreal renovation" returns relevant results
- Qdrant data persists across container restarts

---

## Step 3.2 — Mem0 Structured Entity Extraction

### Goal

A Mem0 sidecar automatically extracts people, projects, preferences, appointments, and other structured facts from every conversation and stores them as queryable entities.

### Files to modify

```
src/lib/dokploy.ts
```

```
services:
  # ... openclaw and qdrant ...

  mem0:
    image: mem0ai/mem0:latest
    restart: unless-stopped
    environment:
      MEM0_VECTOR_STORE: "qdrant"
      QDRANT_URL: "http://qdrant:6333"
      MEM0_COLLECTION: "memories"
      OPENAI_API_KEY: "${openaiApiKey}"  # for embedding generation
    volumes:
      - openclaw-data:/mem0/data
    # Internal only — accessed by openclaw via http://mem0:8080
```

### MCP tools for Mem0

Create `src/lib/mcp/mem0-tool.ts`:

- `memories_get(query)` — retrieve memories relevant to a query
- `memories_add(content)` — manually add a memory
- `memories_list(category?)` — list all memories by category
- `memories_delete(memoryId)` — delete a specific memory
- `memories_update(memoryId, content)` — update a memory

### Categories auto-extracted by Mem0

- `people` — contacts, relationships, roles
- `projects` — ongoing work, clients, deadlines
- `preferences` — user preferences and habits
- `appointments` — recurring events and deadlines
- `tasks` — pending items and commitments
- `products` — things the user sells or manages

### Acceptance criteria

- After conversation mentioning "Sarah is my accountant", `memories_list('people')` returns Sarah's entry
- After "remind me to invoice Marc on the 1st", `memories_list('tasks')` has the recurring task
- Preferences are applied to agent responses (test with tone preference)
- Mem0 data persists across container restarts

---

## Step 3.3 — Memory Dashboard UI

### Goal

Users can see, search, and manage everything their agent remembers. This is the transparency feature that builds trust.

### Files to create

```
src/app/dashboard/memory/page.tsx  src/components/memory/MemoryTimeline.tsx
src/components/memory/EntityBrowser.tsx  src/components/memory/MemorySearch.tsx
```

### API routes

```
// GET  /api/memory/entities?category=&search=  — list extracted entities
// GET  /api/memory/timeline?page=              — chronological memory events
// PATCH /api/memory/entities/[id]              — edit a memory
// DELETE /api/memory/entities/[id]             — delete a memory
// GET  /api/memory/export                      — full memory export as JSON
```

These endpoints proxy to the Mem0 and Qdrant APIs running in the user's container.

### UI layout

```
Memory                                [Export] [Search...]
═══════════════════════════════════════════════════════
[People] [Projects] [Preferences] [Tasks] [Timeline]

People (12)
┌──────────────────────────────────────────────────┐
│ Sarah Tremblay   Accountant · QuickBooks         │
│ +1-514-xxx-xxxx · Last mentioned: 3 days ago     │
│                                        [Edit][×] │
├──────────────────────────────────────────────────┤
│ Marc Dubois      Client · Monthly invoice $3,500 │
│                                        [Edit][×] │
└──────────────────────────────────────────────────┘
```

### Acceptance criteria

- Memory Dashboard shows all extracted entities in categories
- User can edit a memory (corrects the stored value)
- User can delete a memory (removes from Mem0 + Qdrant)
- Export downloads a `.json` file with all memories
- Search works by entity name and content

---

## Step 3.4 — Notion Integration

### Goal

The agent can read and write to the user's Notion workspace: create pages, update databases, query data.

### OAuth setup

Notion uses OAuth 2.0. Create a Notion integration at `notion.so/my-integrations`.

### MCP tools

- `notion_create_page(parentId, title, content)` — create a page
- `notion_update_page(pageId, content)` — update page content
- `notion_query_database(databaseId, filter?, sort?)` — query a database
- `notion_create_database_row(databaseId, properties)` — add a row
- `notion_search(query)` — search workspace

### Settings UI

Add Notion to the integrations page with “Connect Notion” OAuth button.

### Acceptance criteria

- Agent can create a Notion page when asked
- Agent can query a Notion database and return results
- Morning Briefing can optionally pull tasks from a Notion database

---

## Step 3.5 — Google Sheets Integration

### Goal

The agent can read and update Google Sheets — ideal for financial tracking, project management, and reporting.

### Uses existing Google OAuth

Reuse the Google OAuth from Phase 2. Add scope: `spreadsheets`.

### MCP tools

- `sheets_read(spreadsheetId, range)` — read cell range
- `sheets_write(spreadsheetId, range, values)` — write values
- `sheets_append(spreadsheetId, range, values)` — append rows
- `sheets_create(title)` — create new spreadsheet

### Acceptance criteria

- Agent can read data from a Google Sheet and summarize it

- Agent can append a row to a Sheet (e.g., log a daily expense)

---

## Step 3.6 — TickTick Integration

### Goal

The agent can create and manage tasks in TickTick without the user explicitly asking — it notices when tasks arise in conversation and creates them automatically.

### OAuth

TickTick has an OAuth 2.0 API. Register at `developer.ticktick.com`.

### MCP tools

- `tasks_create(title, dueDate?, priority?, listId?)` — create a task
- `tasks_complete(taskId)` — mark complete
- `tasks_list(listId?, filter?)` — list tasks
- `tasks_update(taskId, changes)` — update task

### Auto-task detection

In the system prompt additions (OpenClaw supports per-instance system prompt additions), include:

```
When the user mentions something they need to do, have to remember, or has a dead
create a task in TickTick automatically without asking permission.
Mention briefly that you've added it: "Added to TickTick: [task name]"
```

### Acceptance criteria

- "I need to call the accountant before Friday" → task appears in TickTick automatically
- Agent can list open tasks when asked "what do I have to do this week?"

---

## Phase 3 Definition of Done

- Qdrant running as sidecar, vectors persisting across restarts

- Mem0 extracting entities from conversations
- Memory Dashboard shows entities with edit/delete/export
- Semantic search: “find that renovation project” returns correct results
- Notion: agent can read and create pages
- Google Sheets: agent can read and write cells
- TickTick: agent auto-creates tasks from conversation
- Pro tier unlocked (Qdrant + Mem0 only on Pro+)
- All previous tests still pass

---

---

# PHASE 4 — Skills Marketplace + More Channels

**Goal:** Launch the Skills Marketplace with 10 initial skills. Add WhatsApp configuration in the dashboard. Launch the Builder tier.

**Timeline:** Month 4–5

**Prerequisite:** Phase 3 complete.

---

## Step 4.1 — Skills Marketplace Infrastructure

### Goal

A skills management system: install, configure, enable/disable, and monitor skills from the dashboard.

### Database schema

```
model Skill {
  id          String       @id @default(cuid())
  slug        String       @unique  // 'morning-briefing', 'inbox-zero'
  name        String
  description String
  category    String       // 'core', 'research', 'content', 'developer', 'busine
  tier        String       // 'starter', 'pro', 'builder'
  configSchema Json        // JSON Schema for config form
```

```
  icon        String       // emoji or icon name
  source      String       // 'enclave' | 'clawhub' | 'community'
  createdAt   DateTime     @default(now())
  userSkills  UserSkill[]
}

// Extend UserSkill (from Phase 2):
model UserSkill {
  // ... existing fields ...
  skill       Skill        @relation(fields: [skillId], references: [slug])
  lastRunAt   DateTime?
  lastRunStatus String?    // 'success' | 'failed'
  runCount    Int          @default(0)
}
```

Run: `npx prisma migrate dev --name add_skills_catalog`

Seed the skills catalog with the 10 launch skills:

```
// prisma/seed.ts — add skills seed
const skills = [
  { slug: 'morning-briefing',     name: 'Morning Briefing',     category: 'core',
  { slug: 'evening-winddown',     name: 'Evening Wind-Down',    category: 'core',
  { slug: 'meeting-prep',         name: 'Meeting Prep',         category: 'core',
  { slug: 'meeting-notes',        name: 'Meeting Notes',        category: 'core',
  { slug: 'inbox-zero',           name: 'Inbox Zero',           category: 'core',
  { slug: 'weekly-review',        name: 'Weekly Review',        category: 'core',
  { slug: 'lead-researcher',      name: 'Lead Researcher',      category: 'resear
  { slug: 'competitor-monitor',   name: 'Competitor Monitor',   category: 'resear
  { slug: 'content-repurposer',   name: 'Content Repurposer',   category: 'conten
  { slug: 'client-onboarding',    name: 'Client Onboarding',    category: 'busine
]
```

## Skills Marketplace UI

Create `src/app/dashboard/skills/page.tsx`:

```
Skills Marketplace
═══════════════════════════════════════

[All] [Core] [Research] [Content] [Developer] [Business]

┌──────────────────┐  ┌──────────────────┐
│ ☀️ Morning       │  │ 📋 Meeting Prep  │
│ Briefing         │  │                  │
│ Daily digest     │  │ Auto-prep before │
```

```
│ on Telegram      │  │ calendar events  │
│                  │  │                  │
│ [Installed ✓]    │  │ [Install]        │
└──────────────────┘  └──────────────────┘
```

### Skill configuration modal

When installing a skill with config options, show a modal with the config form generated from `configSchema`.

### Acceptance criteria

- Skills catalog shows all 10 launch skills
- User can install/uninstall a skill
- Tier gating: Pro-only skills show lock icon on Starter plan
- Config form appears for skills that need configuration (e.g., Morning Briefing time)
- Installed skills appear in a "My Skills" section

---

## Step 4.2 — Content Skills Implementation

### Goal

Implement the Content Repurposer and Newsletter Writer skills as fully autonomous agents.

### Content Repurposer

When triggered (manually or via Telegram command), the agent:

1. Takes a URL, blog post text, or document from workspace
2. Generates: Twitter/X thread, LinkedIn post, email newsletter snippet, TikTok script
3. Saves all 4 outputs to workspace as separate files
4. Sends a summary to Telegram for review

Skill config schema:

```
{
  "defaultTone": "professional|casual|witty",
  "twitterMaxTweets": 10,
  "linkedinMaxWords": 300,
```

```
  "outputFolder": "content-repurposed"
}
```

### Newsletter Writer

Cron-based (weekly or on-demand):

1. Read previous newsletters from workspace to avoid repetition
2. Research configured topics via web search
3. Draft newsletter in configured format (Substack/Ghost/plain text)
4. Save draft to workspace + send preview to Telegram

### Acceptance criteria

- "Repurpose this article [URL]" → 4 outputs in workspace within 60 seconds
- Newsletter draft is saved to workspace and previewed in Telegram

---

## Step 4.3 — Research Skills Implementation

### Goal

Implement Lead Researcher and Competitor Monitor as background skills.

### Lead Researcher

Triggered via chat: "Research [company/person name]"

Output saved to workspace as `research/{name}-{date}.md`:

```
# Research Brief: [Name]
Generated: [date]

## Overview
...

## Recent News (last 30 days)
...

## Social Presence
...
```

```
## Suggested Angles
...
```

### Competitor Monitor

Cron: weekly (configurable day/time)

Config:

```
{
  "competitors": ["competitor1.com", "competitor2.com"],
  "trackChanges": ["pricing", "features", "blog"],
  "deliverTo": "telegram"
}
```

Output: weekly diff report saved to workspace + sent to Telegram.

### Acceptance criteria

- Lead research produces a structured brief in workspace within 2 minutes
- Competitor Monitor fires weekly and produces a diff report

---

## Step 4.4 — WhatsApp Channel Setup in Dashboard

### Goal

Users can connect their WhatsApp account from the dashboard settings (not just Telegram).

### How it works

OpenClaw's WhatsApp integration uses Baileys, which requires QR code scanning. The dashboard shows a QR code the user scans with WhatsApp.

### Implementation

1. When user clicks "Connect WhatsApp" in settings:
2. Call OpenClaw gateway API to start WhatsApp auth session
3. OpenClaw returns a QR code (base64 image)
4. Display QR code in a modal — user scans with WhatsApp

5. OpenClaw fires a webhook when pairing completes
6. Update `instance.whatsappConnected = true` in DB

```
// src/app/api/channels/whatsapp/start/route.ts
// POST — initiate WhatsApp pairing, return QR code

// src/app/api/channels/whatsapp/status/route.ts
// GET — polling endpoint for pairing completion
```

### Acceptance criteria

- QR code appears in dashboard within 3 seconds of clicking “Connect WhatsApp”
- After scanning, agent responds on WhatsApp within 30 seconds
- WhatsApp and Telegram share the same agent memory

---

## Step 4.5 — Builder Tier + Multi-Agent Routing

### Goal

Builder tier users ($59/mo) get up to 3 specialized agents with different roles. Each agent has its own memory context and can be routed to different channels.

### Database changes

```
// Extend Instance to support multiple agents:
model AgentProfile {
  id          String   @id @default(cuid())
  userId      String
  user        User     @relation(fields: [userId], references: [id])
  name        String   // "Work Agent", "Personal Assistant"
  role        String   // system prompt role description
  model       String   // LLM model for this agent
  channels    String[] // which channels route to this agent
  soulFile    String?  // custom SOUL.md content
  active      Boolean  @default(true)
  createdAt   DateTime @default(now())
}
```

### Multi-agent routing in OpenClaw config

```
{
  "agents": {
    "routes": [
      { "channel": "telegram", "account": "work_bot", "agent": "work" },
      { "channel": "telegram", "account": "personal_bot", "agent": "personal" },
      { "channel": "whatsapp", "agent": "personal" }
    ],
    "profiles": {
      "work": { "model": "anthropic/claude-opus-4-6", "workspace": "/data/work" }
      "personal": { "model": "anthropic/claude-sonnet-4-6", "workspace": "/data/p
    }
  }
}
```

### Acceptance criteria

- Builder tier users can create up to 3 agent profiles
- Each agent profile has its own name, model, and system role
- Different Telegram bots route to different agents
- Each agent has isolated memory context (separate lossless-claw database paths)

---

## Phase 4 Definition of Done

- Skills Marketplace shows 10 launch skills with install/configure UI
- Tier gating enforced (Pro skills require Pro subscription)
- Content Repurposer and Newsletter Writer working
- Lead Researcher and Competitor Monitor working
- WhatsApp can be connected from the dashboard
- Builder tier: multi-agent routing in dashboard
- Encrypted backup add-on available in billing settings
- All previous tests passing

---

---

# PHASE 5 — Teams Tier + Developer Skills + Mobile

**Goal:** Teams tier for groups, developer-focused skills (Sentry, GitHub, SSH), Voice support, and data portability.

**Timeline:** Month 5–6

**Prerequisite:** Phase 4 complete.

---

## Step 5.1 — Teams Tier

### Database schema

```
model Team {
  id          String       @id @default(cuid())
  name        String
  ownerId     String
  owner       User         @relation("TeamOwner", fields: [ownerId], references:
  members     TeamMember[]
  knowledgeBase WorkspaceFile[] // shared files
  stripeSubId String?      // team subscription
  createdAt   DateTime     @default(now())
}

model TeamMember {
  id      String @id @default(cuid())
  teamId  String
  team    Team   @relation(fields: [teamId], references: [id])
  userId  String
  user    User   @relation(fields: [userId], references: [id])
  role    String // 'owner' | 'admin' | 'member'
  joinedAt DateTime @default(now())

  @@unique([teamId, userId])
}
```

### Shared knowledge base

Team members can designate files as “shared” — they appear in every team member’s agent context via a shared MCP read-only access layer.

### Team admin dashboard

`src/app/dashboard/team/page.tsx`：

- Invite members by email
- Set shared knowledge base files
- View all member agent statuses

### Acceptance criteria

- Team owner can invite 3+ members by email
- Shared knowledge base files are accessible to all member agents
- Team subscription billed per-seat via Stripe
- Admin can remove a member (revokes shared access)

---

## Step 5.2 — Developer Skills

### Sentry → Codex → PR Skill

Requires: GitHub integration (OAuth), Sentry integration (API key).

Workflow:

1. Sentry webhook fires on new error
2. Agent receives error details + stack trace
3. Agent identifies the relevant code file via GitHub API
4. Agent drafts a fix (using code generation model)
5. Agent opens a PR on GitHub with the fix
6. Agent posts a Slack/Telegram notification: “Opened PR #123 for [error]”

### Infra Manager Skill

Requires: SSH key pair per user, stored encrypted in the volume.

- `ssh_run(host, command)` — run a command on a configured server
- `ssh_edit_file(host, path, content)` — edit a file on a remote server
- `nixos_rebuild(host)` — run `nixos-rebuild switch` on a NixOS host

Security requirements:

- SSH private keys stored encrypted with the user's `ENCLAVE_ENCRYPTION_KEY`
- Keys never leave the user's container
- Command allowlist configurable by user

### Deploy Monitor Skill

Integrates with Railway/Render/Fly.io via their webhooks and APIs:

- Monitor deployment status
- Auto-rollback on critical errors (configurable threshold)
- Notify via Telegram on every deploy: success ✓ or failure ✕ with log snippet

### Acceptance criteria

- Sentry error → GitHub PR in < 2 minutes (end-to-end test)
- SSH Manager can run a command on a test server via Telegram
- Deploy Monitor notifies on Railway deployment events

---

## Step 5.3 — Memory Export (Data Portability)

### Goal

Users can download a complete export of everything their agent knows and has said. This fulfills GDPR Article 20 (data portability) and reinforces the privacy promise.

### Export format

```
enclave-export-{userId}-{date}.zip
├── conversations/
│   ├── {sessionId}-{date}.jsonl    — raw messages from lossless-claw
│   └── ...
├── memories/
│   ├── entities.json               — all Mem0 entities
│   ├── vectors-metadata.json       — Qdrant vector metadata (no raw embeddings)
│   └── summaries.json              — lossless-claw DAG summaries
├── workspace/
│   ├── {folder}/{filename}         — all workspace files
```

```
│   └── ...
└── export-manifest.json            — metadata, export date, schema version
```

### API route

```
// POST /api/export/request
// Triggers async export job, returns jobId

// GET  /api/export/[jobId]/status
// Returns: 'pending' | 'processing' | 'ready' | 'expired'

// GET  /api/export/[jobId]/download
// Streams the zip file (available for 24 hours)
```

### Acceptance criteria

- Export request completes within 5 minutes for a typical user
- Downloaded zip contains conversations, memories, and workspace files
- Export is encrypted with the user's key (they can only decrypt if they have the key)
- Export job status visible in dashboard under Settings > Data

---

## Step 5.4 — Voice Channel (Talk Mode)

### Goal

Users can speak to their agent via the Enclave dashboard using browser microphone.

### Implementation

OpenClaw supports Talk Mode natively. Wire it to the web chat interface:

```
// Add to ChatEmbed.tsx:
// A microphone button that activates Talk Mode
// Uses the OpenClaw Gateway WebSocket's voice endpoint
// Text-to-speech response plays in browser
```

Requirements:

- ElevenLabs API key for high-quality TTS (configurable by user)

- Falls back to browser TTS if no ElevenLabs key
- Wake word support (optional, configured in settings)

### Acceptance criteria

- Microphone button in web chat activates voice input
- Agent response plays as audio in browser
- Voice input transcript shows in chat history

---

## Phase 5 Definition of Done

- Teams tier: invite members, shared knowledge base, per-seat billing
- Sentry → PR skill working end-to-end
- SSH Infra Manager working
- Memory export delivers complete data package
- Voice input working in web chat
- Signal + Matrix channels configurable from dashboard
- All previous tests passing

---

---

# CROSS-CUTTING REQUIREMENTS

These apply to every phase. Do not skip any of these.

---

## Security

- **Never log tokens, API keys, or conversation content** — audit all `console.log` calls before each phase ships
- **All integration OAuth tokens encrypted at rest** — use `src/lib/crypto.ts` AES-256-GCM
- **Rate limiting on all API routes** — use `src/lib/ratelimit.ts` (implement if not exists)

- **CSRF protection** — NextAuth handles this for form submissions; add `X-Requested-With` check for API routes
- **Content Security Policy headers** — add in `next.config.ts`
- **Workspace files: validate mime type server-side** — never trust client-provided MIME type

## Billing Enforcement

Every protected feature must check subscription tier:

```
// src/lib/auth.ts — add helper:
export async function requireTier(
  userId: string,
  minTier: 'starter' | 'pro' | 'builder' | 'teams'
): Promise<void> {
  const sub = await getSubscription(userId)
  const tiers = ['starter', 'pro', 'builder', 'teams']
  if (tiers.indexOf(sub.tier) < tiers.indexOf(minTier)) {
    throw new Error(`UPGRADE_REQUIRED:${minTier}`)
  }
}
```

## Error Handling

- All API routes return consistent error shape: `{ error: string, code: string }`
- All provisioning errors set `instance.status = 'failed'` (never leave as `'provisioning'`)
- All MCP tool failures are logged to the activity log (not surfaced raw to the user)

## i18n

- All new UI strings added to both `src/i18n/messages/en.json` and `src/i18n/messages/fr.json`
- No hardcoded English strings in components

## Testing

- Every new API route gets at least one Vitest integration test

- Every new Prisma model gets schema validation tests
- Playwright E2E tests for: signup flow, workspace upload, skills install, Google OAuth (mocked)

## Performance

- Workspace file list: paginate at 50 items, use cursor pagination
- Memory Dashboard: paginate entities at 20 per page
- All dashboard data fetching: use SWR or React Query with 30s stale time
- No N+1 queries: use Prisma `include` or batch queries

---

## Environment Variables Required Per Phase

```
# Phase 1 (add)
TOKEN_ENCRYPTION_KEY=          # openssl rand -hex 32

# Phase 2 (add)
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
GOOGLE_REDIRECT_URI=https://nestai.nickybruno.com/api/integrations/google/callbac

# Phase 3 (add)
# No new platform-level vars — Qdrant + Mem0 run in user containers

# Phase 4 (add)
TICKTICK_CLIENT_ID=
TICKTICK_CLIENT_SECRET=
NOTION_CLIENT_ID=
NOTION_CLIENT_SECRET=

# Phase 5 (add)
SENTRY_CLIENT_ID=             # for Sentry integration OAuth
GITHUB_CLIENT_ID=
GITHUB_CLIENT_SECRET=
ELEVENLABS_API_KEY=           # optional, for TTS
```

---

## File Structure After All Phases

```
src/
├── app/
│   ├── (auth)/login + register         ← Phase 1 ✓ exists
│   ├── dashboard/
│   │   ├── page.tsx                    ← redirect to /chat
│   │   ├── chat/page.tsx               ← Phase 2
│   │   ├── workspace/page.tsx          ← Phase 2
│   │   ├── memory/page.tsx             ← Phase 3
│   │   ├── skills/page.tsx             ← Phase 4
│   │   ├── team/page.tsx               ← Phase 5
│   │   └── settings/
│   │       ├── integrations/page.tsx   ← Phase 2
│   │       ├── channels/page.tsx       ← Phase 4
│   │       ├── skills/page.tsx         ← Phase 2 (morning briefing)
│   │       └── data/page.tsx           ← Phase 5 (export)
│   ├── api/
│   │   ├── instance/status/route.ts    ← Phase 1
│   │   ├── workspace/                  ← Phase 2
│   │   ├── integrations/               ← Phase 2
│   │   ├── channels/                   ← Phase 4
│   │   ├── memory/                     ← Phase 3
│   │   ├── skills/                     ← Phase 4
│   │   ├── team/                       ← Phase 5
│   │   └── export/                     ← Phase 5
├── lib/
│   ├── dokploy.ts                      ← updated each phase
│   ├── crypto.ts                       ← Phase 1
│   ├── mcp/
│   │   ├── workspace-tool.ts           ← Phase 2
│   │   ├── google-calendar-tool.ts     ← Phase 2
│   │   ├── gmail-tool.ts               ← Phase 2
│   │   ├── qdrant-tool.ts              ← Phase 3
│   │   ├── mem0-tool.ts                ← Phase 3
│   │   ├── notion-tool.ts              ← Phase 3
│   │   ├── sheets-tool.ts              ← Phase 3
│   │   └── ticktick-tool.ts            ← Phase 3
│   └── skills/
│       ├── morning-briefing.ts         ← Phase 2
│       ├── content-repurposer.ts       ← Phase 4
│       ├── lead-researcher.ts          ← Phase 4
│       └── sentry-to-pr.ts             ← Phase 5
└── components/
    ├── workspace/                       ← Phase 2
    ├── memory/                          ← Phase 3
    └── skills/                          ← Phase 4
```

## When to Pause and Ask

Only stop for human input when:

1. A Stripe product/price ID needs to be manually created in the Stripe dashboard
2. A third-party OAuth app needs to be registered (Google, Notion, GitHub) and credentials are not in `.env.local`
3. The Dokploy API responds with an undocumented error that isn't in `src/lib/dokploy-api.ts`
4. An OpenClaw config option behaves differently from the docs and you cannot determine the correct value from testing

For all other decisions: choose the most robust option, document the choice in a comment, and continue.